

CO4_AT1_SIMATS_Compiler

[← Back](#)

QUESTIONS

Q1 ✓
Multiple Inheritance in Smart Buildings
10 marks

Q2 ✓
Hierarchical Inheritance for EVs
10 marks

Q3 ✓
Function Overriding in Green Transport
10 marks

Q4 ✓
Multilevel Inheritance for Circular Economy
10 marks

Q5 ✓
Abstract Class for Carbon Footprint
10 marks

Q5 Abstract Class for Carbon Footprint

10 marks

A carbon-footprint calculator calculates emissions for different activities such as Transport and Electricity. Analyze how abstract classes and virtual functions ensure flexibility.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class CarbonFootprint {
5 public:
6     virtual void calculateEmission() = 0;
7 };
8
9 class Transport : public CarbonFootprint {
10 public:
11     void calculateEmission() override {
12         cout << "Transport Emission: 120 kg CO2" << endl;
13     }
14 };
15
16 class Electricity : public CarbonFootprint {
17 public:
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

← Back

QUESTIONS

Q1 Multiple Inheritance in Smart Buildings
10 marks

Q2 Hierarchical Inheritance for EVs
10 marks

Q3 Function Overriding in Green Transport
10 marks

Q4 Multilevel Inheritance for Circular Economy
10 marks

Q5 Abstract Class for Carbon Footprint
10 marks

Q4 Multilevel Inheritance for Circular Economy

10 marks

A circular-economy system classifies products through multiple levels such as ElectronicDevice → Smartphone → RecyclableSmartphone. Implement the hierarchy.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class ElectronicDevice {
5 public:
6     void device() {
7         cout << "Product: Electronic Device" << endl;
8     }
9 };
10
11 class Smartphone : public ElectronicDevice {
12 public:
13     void phone() {
14         cout << "Type: Smartphone" << endl;
15     }
16 };
17
```

Input

stdin input

Output

Visible Test Case

Test Case 1

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

CO4_AT1_SIMATS_Compiler

← Back

QUESTIONS

Q1 Multiple Inheritance in Smart Buildings
10 marks

Q2 Hierarchical Inheritance for EVs
10 marks

Q3 Function Overriding in Green Transport
10 marks

Q4 Multilevel Inheritance for Circular Economy
10 marks

Q5 Abstract Class for Carbon Footprint
10 marks

Q4 Multilevel Inheritance for Circular Economy

10 marks

A circular-economy system classifies products through multiple levels such as ElectronicDevice → Smartphone → RecyclableSmartphone. Implement the hierarchy.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class ElectronicDevice {
5 public:
6     void device() {
7         cout << "Product: Electronic Device" << endl;
8     }
9 };
10
11 class Smartphone : public ElectronicDevice {
12 public:
13     void phone() {
14         cout << "Type: Smartphone" << endl;
15     }
16 };
17
```

Input

stdin input

Output

Visible Test Case

Test Case 1

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

CO4_AT1_SIMATS_Compiler

← Back

QUESTIONS

Q1 ✓
Multiple Inheritance in Smart Buildings
10 marks

Q2 ✓
Hierarchical Inheritance for EVs
10 marks

Q3 ✓
Function Overriding in Green Transport
10 marks

Q4 ✓
Multilevel Inheritance for Circular Economy
10 marks

Q5 ✓
Abstract Class for Carbon Footprint
10 marks

5/5 answered

Q3 Function Overriding in Green Transport

10 marks

A green-transport platform calculates environmental impact differently for ElectricBus and DieselBus. Analyze how function overriding improves runtime behavior.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Bus {
5 public:
6     virtual void environmentalImpact() {
7         cout << "Bus environmental impact." << endl;
8     }
9 };
10
11 class ElectricBus : public Bus {
12 public:
13     void environmentalImpact() override {
14         cout << "Electric Bus has low environmental impact." << endl;
15     }
16 };
17
```

Input

stdin input

Output



Visible Test Cases

Test Case 1

Q2 Hierarchical Inheritance for EVs

An EV company manages different vehicle types such as ElectricCar and HybridCar. Analyze how reuse common vehicle and battery properties.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     string type;
7     int battery;
8
9     void getData() {
10         cin >> type >> battery;
11     }
12
13     void display() {
14         cout << "Vehicle Type: " << type << endl;
15         cout << "Battery Capacity: " << battery << " kWh" << endl;
16     }
17 };
```